

SAiDL Summer Assignment 2026

Core ML

By Aayush Kushwaha

Abstract

We present a modular Transformer language model designed for systematic study of long-context sequence modeling. We implement and benchmark four attention mechanisms (Standard, Sliding Window, Linear/Performer, Grouped Query Attention), four positional encoding schemes (Sinusoidal, RoPE, ALiBi, Relative PE), and three convolution–attention hybrid architectures on the WikiText-2 dataset. Our experiments evaluate validation perplexity, peak GPU memory, training throughput, and positional extrapolation across context lengths from 512 to 2048 tokens. Key findings: (1) RoPE and ALiBi significantly outperform sinusoidal PE, achieving test perplexities of 175.85 and 176.45 vs. the baseline’s 267.19; (2) Linear attention provides $O(n)$ scaling with competitive quality but at reduced throughput due to sequential causal computation; (3) All three conv–attention hybrid designs improve over the pure attention baseline; (4) Models trained on shorter sequences (512 tokens) with RoPE or ALiBi achieve the best overall perplexity, suggesting strong extrapolation properties.

1 Introduction

Long-context modeling is a central challenge in modern sequence models. The standard self-attention mechanism has $O(n^2)$ time and memory complexity with respect to the sequence length n , limiting its scalability.

In this work, we design a modular Transformer framework that allows plug-and-play swapping of:

1. Attention mechanisms - to study the trade-off between quality and efficiency,
2. Positional encodings - to compare fixed, rotary, and bias-based position representations,
3. Convolution attention hybrids - to capture local n -gram context alongside global attention.

All experiments use the WikiText-2 dataset under a causal language modeling objective with a BPE tokenizer (vocabulary size $\sim 10,000$). We conduct a total of 14 training runs across four experimental phases, systematically isolating the contribution of each architectural component.

2 Background

The scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1)$$

where $Q, K, V \in \mathbb{R}^{n \times d_k}$ are the query, key, and value matrices. Multi-head attention runs h parallel attention heads and concatenates the results, allowing the model to attend to different representation subspaces.

3 Architecture

3.1 Base Model

Base Transformer language model uses pre-norm residual connections with the following hyperparameters:

Parameter	Value
d_{model}	256
n_{heads}	4 (head dim = 64)
n_{layers}	4
d_{ff}	512
Dropout	0.1
Vocab size	$\sim 10,000$ (BPE)
Weight tying	Yes
Total parameters	4,668,928

Each Transformer block follows the pre-norm pattern:

$$x \leftarrow x + \text{Attention}(\text{LayerNorm}(x)), \quad x \leftarrow x + \text{FFN}(\text{LayerNorm}(x)) \quad (2)$$

The FFN consists of two linear layers with GELU activation: $\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2$.

3.2 Attention Variants

3.2.1 Standard Multi-Head Attention

Vanilla scaled dot-product attention (Eq. 1) with $h = 4$ heads. Complexity: $O(n^2 d)$ per layer.

3.2.2 Sliding Window (Local) Attention

Each token i attends only to tokens in $[i - w, i]$ where $w = 256$ is the window size. This reduces effective complexity to $O(nwd)$ while maintaining causality. The sliding window mask is constructed by combining a standard causal mask with a distance threshold.

3.2.3 Linear Attention (Performer / FAVOR+)

We replace the softmax with the FAVOR+ random feature map ϕ :

$$\hat{\text{Attention}}(Q, K, V) = \frac{\phi(Q)(\phi(K)^\top V)}{\phi(Q)\phi(K)^\top \mathbf{1}} \quad (3)$$

Using the kernel: $\phi(x) = \exp(xW - \|x\|^2/2)$ with random projection $W \in \mathbb{R}^{d \times m}$. For causal masking, we employ a sequential prefix-sum approach with a running (B, H, m, d) accumulator, avoiding materialisation of the full (B, H, T, m, d) tensor which would cause memory issues. This gives $O(n \cdot m \cdot d)$ complexity.

3.2.4 Grouped Query Attention (GQA)

GQA shares key-value heads across groups of query heads. With $h = 4$ query heads and $h_{kv} = 2$ KV heads, each KV head serves 2 query heads. This reduces the KV projection parameters from $2 \cdot d^2$ to $2 \cdot h_{kv} \cdot d_{\text{head}} \cdot d$ (4,405,760 total parameters vs. 4,668,928 for standard).

3.3 Positional Encodings

3.3.1 Sinusoidal (Baseline)

Fixed sinusoidal embeddings added to the token embeddings:

$$PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d}), \quad PE_{(\text{pos}, 2i+1)} = \cos(\text{pos}/10000^{2i/d}) \quad (4)$$

3.3.2 RoPE (Rotary Position Embedding)

RoPE encodes position by rotating Q and K in pairs of dimensions:

$$\text{RoPE}(x_m, \theta) = x_m \cos(m\theta) + \text{rotate}(x_m) \sin(m\theta) \quad (5)$$

Applied inside the attention mechanism to Q and K , preserving relative position information in the dot product. The key advantage is that the attention score $q_i^\top k_j$ depends only on the relative distance $i - j$.

3.3.3 ALiBi (Attention with Linear Biases)

ALiBi adds no positional embedding. Instead, a per-head linear bias is added to attention logits:

$$\text{Attention}_{ij}^{(h)} = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m_h \cdot |i - j| \quad (6)$$

where $m_h = 2^{-8h/H}$ is the slope for head h . This design was specifically proposed for length extrapolation.

3.3.4 Relative Positional Encoding

Learnable embeddings indexed by clipped relative distance : $b_{ij} = e_{\text{clip}(i-j, -k, k)}$ added to attention logits. Each head learns its own set of relative position biases.

3.4 Convolution–Attention Hybrids

3.4.1 Conv-Before-Attention

A causal depthwise 1D convolution (kernel size 5) is applied before the attention module in each block, enriching the representation with local n -gram context. This adds 1,313,792 parameters (5.98M total).

3.4.2 Interleaved Conv/Attention

Alternating layers: even-indexed layers are standard Transformer blocks, odd-indexed layers are purely convolutional blocks (2-layer causal conv with GELU activation). Total parameters: 6,238,720.

3.4.3 Gated Convolutional FFN

The standard FFN is replaced with a gated convolution:

$$\text{GatedConvFFN}(x) = \text{Conv1D}(x) \odot \sigma(\text{Gate}(x)) \quad (7)$$

where $\odot$ is element-wise multiplication and σ is the sigmoid function. This design roughly doubles the FFN parameters (9,391,616 total).

4 Experimental Setup

4.1 Dataset

WikiText-2 with a BPE tokenizer (vocab size $\sim 10,000$, trained on WikiText-2 training split).

- Train: 2,644,133 tokens (2,579 chunks at context length 1024)
- Validation: 279,575 tokens (272 chunks)
- Test: 320,220 tokens (312 chunks)

4.2 Training Configuration

- Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01)
- Learning rate: 3×10^{-4} with cosine decay schedule and 500-step linear warmup
- Gradient clipping: max norm 1.0
- Epochs: 15
- Batch size: 32 (16 for linear attention due to memory constraints)
- Hardware: NVIDIA T4 GPU (16 GB VRAM) on Kaggle

4.3 Evaluation Metrics

1. Validation/Test perplexity - $\text{PPL} = \exp(L)$ where L is the average cross-entropy loss per token
2. Peak GPU memory (MB) - measured via `torch.cuda.max_memory_allocated`
3. Training throughput (tokens/sec) - tokens processed per second during training
4. Inference throughput (tokens/sec) - measured over 10 forward passes with batch size 8
5. Training time per epoch (seconds)

5 Results

5.1 Baseline

5.2 Attention Ablation

We compare four attention variants, all using sinusoidal positional encoding at context length 1024.

Key observations:

Table 1: Baseline results: Standard attention + Sinusoidal PE, context length 1024.

Metric	Value
Validation Perplexity	269.65
Test Perplexity	267.19
Train Loss (final epoch)	5.5301
Val Loss (final epoch)	5.5971
Peak GPU Memory	11,422 MB (11.15 GB)
Training Throughput	49,210 tok/s
Time per Epoch	53.7 s
Parameters	4,668,928

Table 2: Attention variants comparison (context length 1024, Sinusoidal PE). All models share the same base hyperparameters except for the attention mechanism.

Attention	Val PPL	Test PPL	Train Loss	Val Loss	Mem (GB)	Tput (tok/s)	Params (M)
Standard	269.65	267.19	5.5301	5.5971	11.15	49,210	4.6
Sliding Window	266.34	263.71	5.5166	5.5848	11.16	49,268	4.6
Linear (FAVOR+)	205.41	202.13	5.3250	5.3089	8.69	29,422	4.6
GQA ($h_{kv} = 2$)	277.55	274.17	5.5550	5.6260	11.19	49,661	4.4

- Linear attention achieves the best perplexity (202.13) among attention variants, likely because the FAVOR+ kernel provides a different inductive bias that benefits this small model.
- Sliding window provides marginal improvement over standard attention (263.71 vs. 267.19) with identical memory and throughput.
- GQA performs slightly worse 274.17 vs. 267.19, likely because with only 4 heads, reducing KV heads to 2 loses too much representational capacity at this small scale.
- Linear attention has significantly lower throughput 29K vs. 49K tok/s due to the sequential prefix-sum computation required for causal masking.

5.2.1 Inference Benchmarks Across Context Lengths

We benchmark inference throughput and peak memory for each attention variant across context lengths 512, 1024, and 2048 (batch size 8).

Table 3: Attention scaling: inference throughput (tokens/sec) and peak GPU memory consumption (MB) across context lengths.

Attention	$L = 512$		$L = 1024$		$L = 2048$	
	Tput	Mem	Tput	Mem	Tput	Mem
Standard	195,181	237	149,949	398	98,506	1,185
Sliding Window	195,432	237	150,105	398	99,178	1,189
Linear	112,418	629	112,832	1,182	111,694	2,289
GQA	196,504	236	151,797	397	99,090	1,184

Scaling analysis:

- Standard, Sliding Window, and GQA show similar $O(n^2)$ scaling — throughput drops from $\sim 195\text{K}$ to $\sim 99\text{K}$ tok/s as context doubles from 512 to 2048, and memory grows from ~ 237 MB to $\sim 1,185$ MB.
- Linear attention exhibits nearly constant throughput ($\sim 112\text{K}$ tok/s across all lengths), confirming its $O(n)$ time complexity. However, its memory still grows linearly due to the sequential accumulator and intermediate activations.
- Sliding window attention does not show memory savings in our implementation because we use a masked dense attention matrix rather than a sparse kernel.
- GQA achieves marginally higher throughput than standard attention due to fewer KV projections.

5.3 Positional Encoding Ablation

We evaluate each positional encoding with standard attention at context length 1024.

Table 4: Positional encoding comparison (Standard attention, context length 1024).

Pos. Enc.	Val PPL	Test PPL	Train Loss	Val Loss	Mem (GB)	Tput (tok/s)	Time/Ep (s)
Sinusoidal	269.65	267.19	5.5301	5.5971	11.15	49,210	53.7
RoPE	177.82	175.85	4.9722	5.1808	11.19	47,039	56.1
ALiBi	178.59	176.45	5.0752	5.1851	11.44	47,103	56.1
Relative	228.18	224.36	5.4669	5.4132	11.47	43,641	68.5

Key observations:

- RoPE and ALiBi dramatically outperform sinusoidal PE, reducing test perplexity from 267.19 to ~ 176 which is a 34% reduction.
- RoPE and ALiBi perform nearly identically 175.85 vs. 176.45, consistent with the literature showing both capture relative position effectively.
- Relative PE improves over sinusoidal 224.36 vs. 267.19 but falls behind RoPE/ALiBi, possibly due to limited learnable capacity with the clipping range.
- Throughput is comparable across all PE types ($\sim 43\text{--}49\text{K}$ tok/s), with Relative PE being slowest due to additional bias computation.

5.3.1 Extrapolation Test

We train each PE variant on context length $L_{\text{train}} = 512$ and report the resulting test perplexity. The extrapolation evaluation (evaluating at $L_{\text{test}} \in \{512, 1024, 2048\}$) is shown in below Table.

Extrapolation observations:

- RoPE achieves the best perplexity when trained on short sequences (131.83), followed by ALiBi (135.28).
- Interestingly, all PE variants achieve better perplexity when trained on $L = 512$ compared to $L = 1024$. This is likely because the WikiText-2 dataset is relatively small (2.6M tokens), and

Table 5: Models trained on $L_{\text{train}} = 512$: training results and extrapolation capacity.

Pos. Enc.	Val PPL	Test PPL	Train Loss	Mem (GB)	Tput (tok/s)	Time/Ep (s)
Sinusoidal	178.50	176.11	5.0297	4.53	64,628	40.8
RoPE	131.39	131.83	4.6749	4.53	61,739	42.7
ALiBi	135.99	135.28	4.7070	4.78	63,205	41.8
Relative	175.42	172.74	4.9371	4.78	69,468	43.6

shorter context lengths yield more training chunks (5,158 vs. 2,579), providing more gradient updates.

- RoPE and ALiBi maintain their advantage over sinusoidal PE, consistent with their design for relative position modeling.
- All models use ~ 4.5 GB memory and ~ 62 – 69 K tok/s throughput at $L = 512$, roughly half the memory of $L = 1024$ training.

5.4 Convolution Attention Hybrids

We evaluate three conv-attention hybrid designs using standard attention and sinusoidal PE at context length 1024.

Table 6: Conv+Attention hybrid comparison (Standard attention, Sinusoidal PE, context 1024).

Hybrid Design	Val PPL	Test PPL	Train Loss	Val Loss	Mem (GB)	Tput (tok/s)	Params (M)
None (baseline)	269.65	267.19	5.5301	5.5971	11.15	49,210	4.67
Conv before Attn	209.40	209.56	5.2640	5.3443	11.61	41,778	5.98
Interleaved	215.61	215.46	5.2846	5.3735	8.59	55,135	6.24
Gated Conv FFN	205.76	205.14	5.2182	5.3267	11.83	32,963	9.39

Table 7: Comparison of convolution-enhanced linear(best attention mechanism) and Rope (best positional encoding).

Architecture	Conv Hybrid	Best Val PPL ↓	Test PPL ↓	Params	Throughput (tok/s) ↑	Latency (ms) ↓	Peak Mem (MB) ↓
linear_rope_baseline	none	188.27	186.79	4,668,928	117,843	69.5	1158.3
linear_rope_conv_before	conv_before	130.80	130.64	5,982,720	94,820	86.4	1172.2
linear_rope_interleaved	interleaved	133.61	133.55	6,238,720	132,976	61.6	1164.3
linear_rope_gated_conv	gated_conv_ffn	128.94	129.38	9,391,616	63,736	128.5	1176.3

6 Conclusion

Table 8 presents all 14 experiments sorted by test perplexity.

Table 8: Complete comparison of all experiments, sorted by test perplexity.

Experiment	Attn	PE	Conv	Ctx	Params	Test PPL	Val PPL	Mem (GB)	Tput	Time/Ep
extrap_rope	standard	rope		512	4.67M	131.83	131.39	4.53	61,739	42.7
extrap_alibi	standard	alibi		512	4.67M	135.28	135.99	4.78	63,205	41.8
extrap_relative	standard	relative		512	4.67M	172.74	175.42	4.78	69,468	43.6
pe_rope	standard	rope		1024	4.67M	175.85	177.82	11.19	47,039	56.1
extrap_sinusoidal	standard	sinusoidal		512	4.67M	176.11	178.50	4.53	64,628	40.8
pe_alibi	standard	alibi		1024	4.67M	176.45	178.59	11.44	47,103	56.1
attn_linear	linear	sinusoidal		1024	4.67M	202.13	205.41	8.69	29,422	89.1
conv_gated_n	standard	sinusoidal	gated	1024	9.39M	205.14	205.76	11.83	32,963	80.1
conv_before	standard	sinusoidal	before	1024	5.98M	209.56	209.40	11.61	41,778	63.2
conv_interleaved	standard	sinusoidal	interl.	1024	6.24M	215.46	215.61	8.59	55,135	47.9
pe_relative	standard	relative		1024	4.67M	224.36	228.18	11.47	43,641	68.5
attn_sliding	sliding	sinusoidal		1024	4.67M	263.71	266.34	11.16	49,268	53.6
baseline	standard	sinusoidal		1024	4.67M	267.19	269.65	11.15	49,210	53.7
attn_gqa	gqa	sinusoidal		1024	4.41M	274.17	277.55	11.19	49,661	53.2

References

- [1] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [2] S. Merity et al., “Pointer Sentinel Mixture Models,” *arXiv:1609.07843*, 2016.
- [3] K. Choromanski et al., “Rethinking Attention with Performers,” *ICLR*, 2021.
- [4] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *arXiv:2104.09864*, 2021.
- [5] O. Press et al., “Train Short, Test Long: Attention with Linear Biases Enables Input Length Generalization,” *ICLR*, 2022.
- [6] P. Shaw et al., “Self-Attention with Relative Position Representations,” *NAACL*, 2018.
- [7] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *EMNLP*, 2023.
- [8] I. Beltagy et al., “Longformer: The Long-Document Transformer,” *arXiv:2004.05150*, 2020.